

的实际使用之间的一致性。

情况 2：只有初始化 inode 的操作保存在设备中。同样，由于增加目录项的操作并没有保存在存储设备中，新创建的文件无法在文件系统中被观测到。同时，由于 inode 的分配信息未被修改，后续的创建文件操作依然可以使用该 inode 结构，因此并未产生空间泄漏。这种情况的出现并不会造成文件系统的一致性问题。

情况 3：只有增加目录项的操作保存在设备中。由于增加目录项的操作被持久化在存储设备中，当文件系统遍历该目录时，能够看到新文件对应的文件名和 inode 号。然而由于该 inode 结构中的数据未被初始化，文件系统尝试访问该 inode 时，会访问到未初始化的数据，从而产生错误。同时，由于该 inode 号对应的分配信息未被持久化，在后续的文件操作中，该 inode 可能会被再次分配给其他文件。这将导致该 inode 错误地被两个不同的文件使用，造成数据丢失、泄露和被篡改等问题。这种情况违反了 inode 分配信息、inode 结构与目录项中所保存的 inode 号之间的一致性，即所有目录项中所保存的 inode 号皆应被分配且初始化。

情况 4：只有分配 inode 的操作未保存在设备中。该情况下，由于分配 inode 的信息未被持久化，会造成情况 3 中的目录项错误地指向未分配 inode 结构的情况，从而造成正确性和安全性等问题。

情况 5：只有初始化 inode 的操作未保存在设备中。与情况 3 相似，如果只有 inode 的初始化操作未被持久化，则文件系统在后续访问该文件时，会访问到未初始化的数据，从而产生错误。

情况 6：只有增加目录项的操作未保存在设备中。与情况 1 类似，当增加目录项的操作未被持久化时，会造成 inode 资源泄露的问题。

表 12.1 中给出了上述六种情况的汇总。在以上的介绍中，我们使用一个简化后的文件创建过程作为示例。实际的文件创建过程一般还涉及多种其他结构中的数据，如父目录文件的访问和修改时间，系统占用的 inode 数量统计，以及在增加目录项时需要进行的额外数据块分配。为了避免在更新这些结构时造成一致性的问题，文件系统采用多种方法保持崩溃一致性。接下来我们将一一介绍这些技术。

表 12.1 崩溃发生时的多种情况

崩溃发生时 的情况	文件系统结构			是否影响使用	典型问题
	inode 位图	inode 结构	目录项		
情况 1	持久	未持久	未持久	否	资源泄露
情况 2	未持久	持久	未持久	否	无
情况 3	未持久	未持久	持久	是	信息错乱
情况 4	未持久	持久	持久	是	信息错乱
情况 5	持久	未持久	持久	是	信息错乱
情况 6	持久	持久	未持久	否	资源泄露

12.2 同步写入与文件系统一致性检查

本节主要知识点

- 文件系统的同步写入有什么问题？
- 文件系统如何检查和修复其保存的数据与元数据的不一致？

12.2.1 同步写入

对于一个文件系统操作，文件系统会按照一定的顺序执行其中的各个步骤。如在创建文件的过程中，文件系统一般会先分配 inode，再进行初始化，最后在父目录中增加目录项。为了尽量避免文件系统在崩溃后产生一致性问题，文件系统可以使用同步写入保证修改按序完成。换句话说，文件系统每执行完一个步骤，就需要马上将其对应的修改写入存储设备中。在确认修改已经持久化在存储设备中之后，文件系统再执行下一步修改。这样虽然不能保证一个操作中的多个步骤的原子完成，但至少保证了各个修改按照既定顺序进行持久化。通过合理安排一个文件操作中各个步骤的顺序，可以避免出现指向未初始化数据等比较严重的不一致情况^①。

然而，如果严格按照顺序进行操作，文件系统的每步操作都需要访问存储设备。存储设备的访问速度一般要远小于处理器计算的速度和访问内存的速度。例如，访问一次固态硬盘需要十几到几十微秒，访问机械磁盘需要若干毫秒，而访问一次内存只需要 100 纳秒左右。如果文件系统每步操作都访问存储设备，文件系统的性能将受限于存储设备的访问速度。为了避免访问存储设备成为性能瓶颈，一些文件系统并不保证对所有数据结构上的写入都严格按照顺序进行。通过将向存储设备的写入请求推迟、重新排序、合并等方法，可以减少对存储设备的访问，提高文件系统性能。举例来说，在为文件分配数据空间时，需要分配数据块。如果严格按照同步写入的方式，在分配结果返回前，文件系统需要将分配信息的更改写入存储设备中。因此，当连续分配两个块得到 X 和 $X+1$ 时，会在每次分配时产生一次设备写入。由于表示块 X 和块 $X+1$ 的位大概率在同一个存储块中，如果推迟对存储设备的写入，则这两个分配引发的写入请求可以被合并成一个，从而减少写入请求以提升性能。虽然这种方式可以提升文件系统的性能，但在发生掉电或崩溃等事件时，存储设备中保存的信息有更大的概率出现不一致的情况。例如，在分配数据块的例子中，如果发生崩溃，存储设备中保存的数据可能显示数据块已被某个文件使用，但是分配信息中显示该数据块是未分配状态。

^① 在 12.5 节介绍 Soft updates 时会有更加详细的例子。

12.2.2 文件系统一致性检查

由于掉电、崩溃和硬件故障（如比特翻转）等情况，存储设备上保存的文件系统信息可能发生不一致。严重的情况下，整个文件系统将无法被识别和挂载。为了能够检查并修复文件系统的一致性，文件系统一般会附带一个工具，称为 fsck。该工具对文件系统所管理的存储设备进行扫描，通过比对存储设备中实际存储的数据和元数据，检查是否存在一致性问题。当检测到一致性问题时，fsck 会尝试进行修复。该工具不通过目标文件系统提供的接口访问数据，而是根据文件系统的存储布局格式直接访问存储设备，因而可以在文件系统未被挂载时使用。由于 fsck 与具体的文件系统存储布局相关，不同文件系统会提供不同的文件系统检查工具。例如 Ext 系列文件系统^①的检查工具为 e2fsck^[1]，XFS 的文件系统检查工具为 xfs_repair。

文件系统一致性检查一般包括以下几类：

- 合理性检查。文件系统各结构中保存的部分信息是具有特定含义的，因而其值应该保持在合理的范围内。例如，超级块中记录的文件系统的总大小不应超过设备总大小，目录项中记录的 inode 号不应超过文件系统中 inode 的数量，文件数据索引结构中记录的块号不应超过文件系统的数据块总数。
- 资源分配的一致性检查。文件系统需要记录多种资源（如空间和 inode）的分配情况。这些记录的分配情况应该与对应资源的实际使用情况相同。例如，若一个数据块被标记为已经分配，则其应该能够通过某个文件的数据索引进行访问；若一个数据块被标记为空闲，则其不应出现在任何一个文件的索引结构中。
- 内在不变量检查。文件系统还存在一些内在的不变关系需要被保持。举例来说，文件 inode 中记录的链接数，应该与指向该文件的目录项的个数相同。

为了更好地了解 fsck 是如何工作的，下面将以 Ext 系列文件系统的检查工具 e2fsck 为例，介绍 fsck 如何对不同结构进行检查。第 11 章介绍的基于 inode 的文件系统与 Ext 系列文件系统在结构上有很多相似之处，因此下文将直接使用第 11 章中介绍的概念。

总的来说，e2fsck 将对文件系统的检查分为多个步骤进行，每个步骤关注一部分结构的一致性并进行检查。在 e2fsck 中，检查的过程可以分为以下五个步骤^[2]。

1. 扫描和检查所有的 inode 和索引块。如果一个块被多处引用，则根据元数据保留一个正确的引用。
2. 分别检查各个目录。
3. 检查目录之间的连接性，消除目录之间的循环包含关系，将无法从根到达的目录放入 lost+found 目录中。
4. 检查 inode 的链接数，移除或移动无人认领的 inode。
5. 更新 inode 分配表和块分配表。

① 主要指 Ext2 到 Ext4 文件系统。

这些步骤将全面覆盖文件系统中所有的结构。下面将对不同结构上的检查进行逐一介绍。

超级块

超级块中保存整个文件系统的元数据，对于文件系统的挂载和使用非常重要。比较重要的元数据包括文件的类型（魔法数字）、文件的版本、文件的总大小、各种功能特性的开启状态、逻辑块的大小、根目录位置等。如果这些信息有误，文件系统可能在挂载时因发生错误而失败。即使挂载成功，文件系统也可能因为使用了错误的访问方式而崩溃，甚至会导致文件系统内保存的数据损坏。此外，超级块中还保存了一些统计信息，如空闲 inode 的个数和空闲块的个数等。这些信息与文件系统的状态相对应，因此需要保证这些信息与文件系统的状态一致。

由于超级块的重要性，Ext 文件系统会在存储设备的不同位置保存多份超级块。当其中一个超级块发生故障无法访问时，文件系统可以通过读取其他位置的超级块内容访问文件系统。

在对超级块进行检查时，e2fsck 并不能确定某些元数据是否一定正确（如文件的总大小），但是 e2fsck 依然能对这些元数据信息进行合理性检查，以避免一些不合理的数值。例如文件的总大小一般不会超过设备的总大小。同时，在 e2fsck 做完其他修复后，也会比对和修复超级块中保存的统计信息。例如，在统计了文件系统中使用的 inode 总数之后，计算出空闲 inode 个数，并与超级块中保存的空闲 inode 个数进行对比。

分配信息

文件系统的分配信息保存整个文件系统各种资源的使用情况。比如，inode 分配表会标记每个 inode 的使用情况。当需要使用新的 inode 时，可以从 inode 分配表中获取标记为空闲的 inode 并使用。块分配表与 inode 分配表类似，不过其管理的内容是文件系统中的块。

存储设备中块的实际使用情况和块的分配信息需要保持一致。如果一个块被标记为已分配，可是在文件系统结构中无法找到该块，那么此块的存储空间发生了泄漏。被泄漏的块无法用来保存数据且无法被删除等文件系统操作回收，这会使得整个存储设备的可用容量变小。相反，如果一个块被标记为空闲，却正在被某个文件当作数据块来保存数据，那么当这个数据块再次被分配出去之后，两个文件会错误地共享同一个数据块。这种情况可能会导致重要信息被泄露或者丢失。inode 等结构的分配同样具有相同的一致性问题。

因此，对分配表的一致性检查需要保证分配表中标记为空闲的 inode 和块没有被文件系统使用。为了防止资源泄露，还要保证在分配表中标记为使用的 inode 和块确实在文件系统中被使用。

inode

inode 中保存文件相关的元数据，是文件系统中非常关键的数据结构。inode 中通常

保存文件类型、文件大小、拥有者和拥有组、访问权限、链接数、访问和修改时间等基本元数据，以及文件内容所在位置的索引。对 inode 的一致性检查需要保证这些信息合理且与文件系统中保存的其他信息一致。举例来说，inode 中记录的占用块的数量应与其实际索引的块数量相一致；inode 中保存的链接数应与整个文件系统中保存的指向该 inode 的目录项的个数一致；对于索引中用到的所有索引块和数据块，都应有合理的块号（如块号不应超过文件系统中的最大编号），在分配表中应为被分配的状态，且同一个块不应该被其他 inode 使用。

与超级块一样，inode 中的部分信息是无法通过文件系统中保存的内容进行检查和修复的。这部分信息包括 inode 中保存的拥有者，以及创建、访问和修改时间等。对于这部分元数据，e2fsck 在进行系统一致性检查时，只能进行基本的合理性检查，保证这部分元数据的数值处在合理的范围内。

目录

目录是一种特殊的文件。除了对其 inode 进行检查之外，还应检查其中保存的目录项的一致性。目录项中保存文件名和文件名所对应的 inode 号。在进行检查时，需保证文件名和目录项的长度均在有效且合理的范围内，并且目录项的长度应不小于文件名的长度与 inode 号所占用的空间之和。对于 inode 号，检查时需要保证每个 inode 号在合理且有效的范围内，并且其对应的 inode 在 inode 分配表中标记为已使用。此外，每个目录应至少包含“.”和“..”两个特殊的目录项，且对应的 inode 号应为当前目录和父目录的 inode 号。

目录项的检查与此前介绍的 inode 的检查需要协同进行，以统计每个 inode 的实际链接数，即指向 inode 的目录项个数。在目录项的统计工作完成后，需要按照实际链接数对不同 inode 进行处理。对于链接数为零的 inode，由于从文件系统的根无法访问到该 inode，因此可以在检查后将其删除。对于链接数不为零，却无法从文件系统根出发被访问到的“孤立”inode，在检查后，e2fsck 会将其保存在特殊的目录“lost+found”中。

12.2.3 fsck 的局限和问题

fsck 是在一致性问题发生之后进行补救的工具，虽然具有一定的一致性检查和修复能力，但并不能处理所有的一致性问题。在一些情况下，fsck 需要以交互的方式在用户的指导下完成修复过程；而在另外一些情况下，它对用户丢失的数据也束手无策。

此外，fsck 还存在效率问题。由于内存的容量一般远远小于存储设备的容量，将存储设备中的所有文件数据读取到内存中进行检查是不现实的。因此 fsck 在进行检查和修复时，一般需要多次扫描存储设备以把需要的部分数据加载到内存中进行访问。由于存储设备的访问比较耗时，一次 fsck 经常需要几小时甚至几十小时。另外，在 fsck 进行修复时，如果计算机系统再次发生故障，情况会变得更加复杂。

相对于 fsck 这种在发生损坏后进行检查和修复的方法，预防性地进行文件系统一致

性保护往往能够获得更好的效果。在接下来的几节中，我们将介绍几种用于文件系统的常见系统性方法，包括日志、写时拷贝和 Soft updates。

12.3 原子更新技术：日志

本节主要知识点

- 日志如何保证多个修改同时持久化？
- 日志的方法可以分为哪几类？
- 一个日志的生命周期是怎样的？
- 日志是如何在真实文件系统中应用的？

当有多件事情需要处理时，为了防止遗漏，我们通常会把所有事情先记录在一个待办清单中。这样即使被其他突发情况打断，我们也可以通过检查待办清单，知道有哪些事情是还未进行的，并将其完成。同理，当文件系统想要原子地完成多个修改时，为了防止只完成了部分修改就发生崩溃等情况，其先将这些待做的修改（即“日志”）记录在一个专有的存储区域中。一旦发生故障，文件系统可以通过读取日志将未完成的修改继续完成。

12.3.1 日志机制的原理

在文件系统中使用的日志一般为预写式日志（Write-Ahead Logging, WAL）^[3]。其原本用于在数据库系统中提供数据的原子持久性。正如其名，预写式日志要求在进行修改之前先在日志中记录修改的内容，以便在发生崩溃等情况时通过日志进行数据恢复，保证多个修改的原子性。

日志的生命周期

每一条日志信息从创建到销毁会经过一系列步骤和状态，一般包括：

- 创建：在使用日志之前，需要先进行日志的创建和初始化操作。在此过程中，文件系统会为日志分配内存和存储空间，并初始化维护日志所需的元数据。此后，日志进入写入阶段。
- 写入：在写入阶段，文件系统可以将要进行的操作或操作影响的数据及其所在位置写入日志中。举例来说，如果文件系统想要将位置 0x50 处的数据 A 改成 B，需要在日志中记录“位置 0x50 上的数据从 A 修改为 B”。需要注意的是，由于操作数量较多或者操作的数据量较大，在写入阶段记录的操作信息（即日志内容）往往超出存储设备的原子写入大小，因而无法原子地写入存储设备中。因此，在崩溃后，存储设备中处于写入阶段的日志内容可能是不完整的。为了防止这些不